

Les structures de base MYSQL

Les variables dans MYSQL

Introduction :

Les variables sont utilisées pour stocker des données ou des informations lors de l'exécution d'un programme. Dans MySQL on trouve trois types de variables :

- Les variables définies par l'utilisateur.
- Les variables locales.
- Les variables systèmes.

Les variables définies par l'utilisateur :

Les variables définies par l'utilisateur (user-defined variables) sont aussi appelés des variables de session. Elles sont déclarées à l'aide du symbole "@".

Une variable définie par un utilisateur qui n'est pas visible par d'autres utilisateurs c'est elle est spécifique à une session.

Deux syntaxes possibles pour déclarer les variables utilisateur :

```
# Syntaxe 1 :  
SET @var_name = value;  
  
# Syntaxe 2 :  
SELECT @var_name := value;
```

Remarque : Les variables définies par l'utilisateur ne sont pas sensibles à la casse, ce qui veut dire que **@name** et **@NAME** sont identiques.

Exemple 1 :

```
SET @name = "Ahmed";  
SELECT @name;
```

Exemple 2 :

```
USE w3schools;  
SELECT @max_price := MAX(Price) FROM products;  
  
SELECT ProductID, Unit, Price FROM products WHERE Price = @max_price;
```

Exemple 3 :

Si nous accédons à une variable non déclarée, cela donnera la sortie **NULL**.

```
SELECT @var;
```

Les variables locales :

Une variable locale est un type de variable qui n'est pas préfixé par le symbole "@".

Une variable locale est une variable fortement typée, et sa portée se trouve dans **le bloc de programme stocké** dans lequel elle est déclarée.

MYSQL utilise le mot clé **DECLARE** pour spécifier que la variable est locale. L'instruction **DECLARE** combine également une clause **DEFAULT** pour fournir une valeur par défaut à une variable. Si vous ne fournissez pas la clause **DEFAULT**, elle donnera la valeur initiale **NULL**.

Voici la syntaxe pour créer une variable locale :

```
DECLARE variable_name datatype(size) [DEFAULT default_value];
```

Ici on déclare 3 variables a,b et c e type entier avec comme valeur par défaut 2 :

```
DECLARE a, b, c INT DEFAULT 2;
```

Exemple :

```
DELIMITER //  
CREATE PROCEDURE Test()  
BEGIN  
    DECLARE A INT DEFAULT 100;  
    DECLARE B INT;  
    DECLARE C INT DEFAULT 200;  
    DECLARE D INT;  
    DECLARE E INT;  
    SET B = 90;  
    SET E = A + B;  
    SELECT A, B, C, D, E, A + C;  
END //  
DELIMITER ;  
  
CALL Test();
```

Les variables systèmes :

Les variables système sont des variables prédéfinies qui configurent le fonctionnement de MySQL. Chaque variable système contient une valeur par défaut.

Pour afficher les variables systèmes avec leurs valeurs par défaut :

```
SHOW VARIABLES;
```

Une variable système est précédée par les deux symboles "@@". Par exemple :

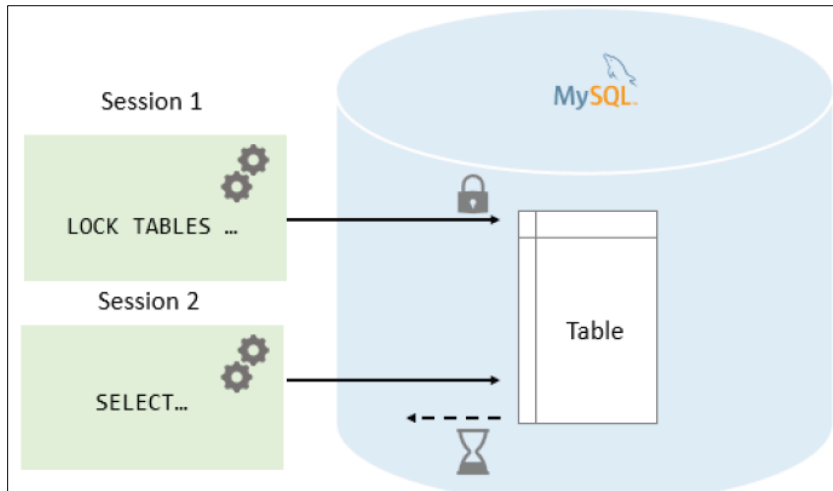
```
SELECT @@version;
```

Verrouillage des tables MYSQL

Définition :

Un **verrou** est un mécanisme associé à une table et qui permet restreindre l'accès aux données de cette table.

Avec les verrous un utilisateur peut empêcher d'autres sessions d'accéder à la même table pendant une période spécifique, et ensuite il peut libérer les verrous pour cette table.



Le verrouillage de table dans MySQL est principalement utilisé lors de l'exécution d'une transaction.

MySQL fournit deux types de verrous (**lock_type**) sur la table :

- **READ** : Ce verrou met la table en lecture et toutes les autres sessions et aussi la session qui 'a défini le verrou ne sont pas en mesure d'écrire des données dans cette table tant que le verrou n'est pas libéré (mais la lecture c'est possible).
- **WRITE** : Ce verrou met la table en écriture ce qui veut dire que la session qui 'a défini le verrou peut accéder et écrire dans cette table par contre toutes les autres sessions ne peuvent pas ni écrire ou accéder aux données de la table tant que le verrou n'est pas libéré.

Voici les syntaxes utilisées pour verrouiller les tables en MYSQL :

```
# Syntaxe 1
LOCK TABLE table_name [AS alias] lock_type;

# Syntaxe 2
LOCK TABLES table1_name [AS alias] lock_type, [table2_name [AS alias] lock_type]...;
```

Voici les syntaxes utilisées pour verrouiller les tables en MYSQL :

```
UNLOCK TABLES;
```

Exemple :

```
USE w3schools;
LOCK TABLE products WRITE;
UNLOCK TABLES;

LOCK TABLES products READ, customers WRITE;
UNLOCK TABLES;
```

Les structures de contrôle conditionnelles MYSQL

L'instruction : IF-THEN

Cette instruction exécute un ensemble de requêtes SQL basées sur certaines conditions ou expressions.

La syntaxe de l'instruction **IF-THEN** est la suivante :

```
IF condition THEN
    statements;
END IF;
```

Exemple :

Dans cet exemple l'instruction **IF-THEN** s'exécute avec une procédure stockée qui permet d'appliquer une remise de 5% si le prix est supérieur à 600 :

```
DELIMITER $$
CREATE PROCEDURE apply_discount1(IN original_price DOUBLE, OUT discount DOUBLE)
BEGIN
    IF original_price > 600 THEN
        SET discount= original_price * 0.05;
    END IF;
    SELECT discount;
END $$
DELIMITER ;

SET @price =1000;
CALL apply_discount1(@price, @discount); # 50

SET @price =500;
CALL apply_discount1(@price, @discount); # NULL
```

L'instruction : IF-THEN-ELSE

Cette instruction exécute aussi des instructions SQL lorsque la condition spécifiée dans le bloc IF n'est pas évaluée comme vraie.

La syntaxe de l'instruction **IF-THEN-ELSE** est la suivante :

```
IF condition THEN
    statements;
ELSE
    else-statements;
END IF;
```

Exemple :

Dans cet exemple l'instruction **IF-THEN-ELSE** s'exécute avec une procédure stockée qui permet d'appliquer une remise de 5% si le prix est supérieur à 600 et dans le cas contraire aucune remise n'est appliquée :

```

DELIMITER $$
CREATE PROCEDURE apply_discount2(IN original_price DOUBLE, OUT discount DOUBLE)
BEGIN
    IF original_price > 600 THEN
        SET discount= original_price * 0.05;
    ELSE
        SET discount= 0;
    END IF;
    SELECT discount;
END $$
DELIMITER ;

SET @price =1000;
CALL apply_discount2(@price, @discount); # 50

SET @price =500;
CALL apply_discount2(@price, @discount); # 0

```

L'instruction : IF-THEN-ELSEIF-ELSE

Cette instruction exécute des instructions basées sur plusieurs conditions.

La syntaxe de l'instruction **IF-THEN-ELSEIF-ELSE** est la suivante :

```

IF condition THEN
    statements;
ELSEIF elseif-condition THEN
    elseif-statements;
...
ELSE
    else-statements;
END IF;

```

Exemple :

Dans cet exemple l'instruction **IF-THEN-ELSEIF-ELSE** s'exécute avec une procédure stockée qui permet d'appliquer une remise de :

- 5% si le prix > 600 e
- 2 % si $100 < \text{prix} \leq 600$
- 0% dans le cas contraire

```

DELIMITER $$
CREATE PROCEDURE apply_discount3(IN original_price DOUBLE, OUT discount DOUBLE)
BEGIN
    IF original_price > 600 THEN
        SET discount= original_price * 0.05;
    ELSEIF original_price > 100 THEN
        SET discount= original_price * 0.02;
    ELSE
        SET discount= 0;
    END IF;
    SELECT discount;
END $$
DELIMITER ;

SET @price =1000;
CALL apply_discount3(@price, @discount); # 50

SET @price =500;
CALL apply_discount3(@price, @discount); # 10

SET @price =80;
CALL apply_discount3(@price, @discount); # 0

```

L'instruction : CASE

Cette instruction est une autre façon pour écrire la logique **IF-THEN-ELSEIF-ELSE**. On trouve deux types d'instructions CASE dans MYSQL :

- Instruction CASE simple
- Instruction CASE recherchée

La syntaxe de l'instruction **CASE** simple dans un bloc est la suivante :

```

CASE variable
    WHEN value1 THEN action1;
    WHEN value2 THEN action2;
    ...
    ELSE result;
END CASE ;

```

La syntaxe de l'instruction **CASE** recherchée dans un bloc est la suivante :

```

CASE
    WHEN condition1 THEN action1;
    WHEN condition2 THEN action2;
    ...
    ELSE result;
END CASE ;

```

La figure suivante montre la différence entre les deux :

```

CASE n
    WHEN 1 THEN Action1;
    WHEN 2 THEN Action2;
    ELSE THEN ActionOther;
END CASE;

```

```

CASE
    WHEN n = 1 THEN Action1;
    WHEN n = 2 THEN Action2;
    WHEN (n > 2 AND n < 6) THEN Action3;
    ELSE THEN ActionOther;
END CASE;

```

Exemple1 :

On va reprendre l'exemple de fonction stockée à l'aide de CASE qui prend comme paramètre le prix d'un article et qui retourne la catégorie de cet article selon le tableau suivant :

Price	Category
price < 50	"Category A"
50 ≤ price < 100	"Category B"
price ≥ 100	"Category C"

```
DELIMITER $$
CREATE FUNCTION IF NOT EXISTS category_price(price DOUBLE)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE category VARCHAR(20);
    CASE
    WHEN price < 50 THEN
        SET category = 'Category A';
    WHEN price >= 50 AND price < 100 THEN
        SET category = 'Category B';
    WHEN price >= 100 THEN
        SET category = 'Category C';
    ELSE
        SET category = 'ERROR';
    END CASE;
    RETURN category;
END $$
DELIMITER ;

SELECT ProductID, Unit, category_price(Price), Price FROM products;
```

Remarque :

Si l'instruction CASE est utilisée directement dans un SELECT en dehors d'un bloc (Fonction ou procédure) alors on utilise la syntaxe suivante :

```
SELECT ...,
CASE ...
    WHEN ... THEN 'result1'
    WHEN ... THEN 'result2'
    ...
    ELSE 'result';
END [AS alias] FROM ...
```

Exemple2 :

Dans la table **order_details** on veut afficher un champ **QuantityText** qui indique si la quantité est égale à 50, ou bien inférieure ou supérieure à 50 :

```
SELECT OrderID, Quantity,
CASE
    WHEN Quantity = 50 THEN 'The quantity is 50'
    WHEN Quantity > 50 THEN 'The quantity is greater than 50'
    ELSE 'The quantity is under 50'
END AS QuantityText FROM order_details;
```

Exemple3 :

L'exemple suivant affiche pour un nombre entre 1 et 10 s'il est premier ou non :

```
SET @number =5;
SELECT
CASE @number
    WHEN 1 THEN 'Le nombre est premier'
    WHEN 2 THEN 'Le nombre est non premier'
    WHEN 3 THEN 'Le nombre est premier'
    WHEN 4 THEN 'Le nombre est non premier'
    WHEN 5 THEN 'Le nombre est premier'
    WHEN 6 THEN 'Le nombre est non premier'
    WHEN 7 THEN 'Le nombre est premier'
    WHEN 8 THEN 'Le nombre est non premier'
    WHEN 9 THEN 'Le nombre est non premier'
    WHEN 10 THEN 'Le nombre est non premier'
    ELSE 'Non inclus'
END AS Result ;
```

Les boucles en MYSQL

LOOP :

L'instruction **LOOP** vous permet d'exécuter une ou plusieurs instructions à plusieurs reprises. Sa syntaxe est :

```
[begin_label:] LOOP
    statement_list
END LOOP [end_label] ;
```

Deux autres instructions sont utilisées avec le LOOP :

- **LEAVE** : permet de sortir immédiatement de la boucle. C'est l'équivalent de **BREAK** dans d'autres langages de programmation comme C, C++, JAVA, PYTHON, ...
- **ITERATE** : permet d'ignorer l'itération de la boucle en cours et démarrer une nouvelle itération. C'est l'équivalent de **CONTINUE** dans d'autres langages de programmation comme C, C++, JAVA, PYTHON, ...

Exemple :

Dans cet exemple on va créer une procédure stockée qui insère les nombres impairs de 1 à 10 dans la table **Test1**.

```
CREATE TABLE Test1(ID INT DEFAULT NULL);

DELIMITER $$
CREATE PROCEDURE insert_odd_number()
BEGIN
    DECLARE x INT DEFAULT 1;
    simple_loop : LOOP
        IF x > 10 THEN
            LEAVE simple_loop;
        END IF;
        IF (x MOD 2 = 0) THEN
            SET x = x + 1;
            ITERATE simple_loop;
        ELSE
            INSERT INTO Test1 VALUES(x);
        END IF;
        SET x = x + 1;
    END LOOP simple_loop;
END
```



```

        SET x = x + 1;
    END IF;
END LOOP simple_loop;
END $$
DELIMITER ;

CALL insert_odd_number();
SELECT * FROM Test1;

```

WHILE :

La boucle **WHILE** est une instruction de boucle qui exécute un bloc de code de manière répétée tant qu'une condition est vraie. Sa syntaxe est :

```

[begin_label:] WHILE condition DO
    statement_list
END WHILE [end_label] ;

```

Exemple :

Dans cet exemple on va créer une procédure stockée qui insère les nombres pairs de 1 à 10 dans la table **Test2**.

```

CREATE TABLE Test2(ID INT DEFAULT NULL);
DELIMITER $$
CREATE PROCEDURE insert_even_number()
BEGIN
    DECLARE x INT DEFAULT 1;
    simple_while : WHILE x <= 10 DO
        IF (x MOD 2 != 0) THEN
            SET x = x + 1;
            ITERATE simple_while;
        ELSE
            INSERT INTO Test2 VALUES(x);
            SET x = x + 1;
        END IF;
    END WHILE simple_while;
END $$
DELIMITER ;

CALL insert_even_number();
SELECT * FROM Test2;

```